

Trabalho em Equipe

Structs, Alocação Dinâmica e Recursão em C

Equipes de 3 integrantes

Prazo de entrega: 27/08/26

1. Apresentação do trabalho

Este trabalho tem como objetivo consolidar três conteúdos já estudados na disciplina: **structs**, **alocação dinâmica de memória** e **funções recursivas**. A equipe deverá desenvolver, em linguagem C, um sistema de **Gerenciamento de Estoque de Produtos** que utilize obrigatoriamente os três conceitos de forma integrada.

2. Descrição do sistema

O sistema simula o controle de estoque de uma loja. Cada produto é representado por uma struct contendo, no mínimo, os seguintes campos:

- **id** (inteiro, único para cada produto)
- **nome** (vetor de caracteres)
- **preco** (float)
- **quantidade** (inteiro)

Os produtos devem ser armazenados em um vetor **alocado dinamicamente** (não pode ser um vetor de tamanho fixo definido em tempo de compilação). Quando o vetor ficar cheio, sua capacidade deve ser aumentada dinamicamente com **realloc**.

3. Requisitos funcionais obrigatórios

O programa deve oferecer um menu em modo texto com, no mínimo, as seguintes operações:

1. **Cadastrar produto** — aloca (ou redimensiona) o vetor dinâmico e insere um novo produto.
2. **Remover produto por id** — remove o produto do vetor e reorganiza os elementos restantes.
3. **Listar todos os produtos** — implementada com uma **função recursiva** (sem uso de laço **for/while** para percorrer o vetor nesta função específica).
4. **Buscar produto por id** — implementada com **busca recursiva** (o vetor deve estar ordenado por id para essa operação).
5. **Ordenar produtos por preço** — implementada com um algoritmo de ordenação.

6. **Calcular o valor total do estoque** (soma de `preco * quantidade` de todos os produtos) — implementada com uma **função recursiva** de soma.
7. **Encerrar o programa** — deve liberar corretamente toda a memória alocada dinamicamente antes de finalizar (uso de `free`).

4. Requisitos técnicos e boas práticas

- O código deve ser organizado em arquivos separados: `produto.h` (interface), `produto.c` (implementação) e `main.c` (menu e uso).
- A struct de `Produto` deve ficar declarada no cabeçalho (`produto.h`).
- Todas as funções recursivas devem ser, de fato, recursivas — não serão aceitas versões iterativas disfarçadas de recursivas.
- O programa não pode apresentar vazamento de memória. Recomenda-se testar com ferramentas como `valgrind` ou a flag `-fsanitize=address` do `gcc`.
- Tratar casos de erro: falha de alocação (`malloc/realloc` retornando `NULL`), id inexistente na busca/remoção, estoque vazio, etc.
- Comentar o código, especialmente as funções recursivas (explicando o caso base e o caso recursivo).

5. Entrega

Ao final dos 8 dias, cada equipe deve entregar:

- Código-fonte completo (arquivos `.h` e `.c`), compilando sem erros nem warnings.
- Um arquivo `README` com: nome dos 3 integrantes, instruções de compilação (ex.: linha de comando `gcc`) e instruções de uso do menu.
- Os arquivos devem ser entregues compactados em um único `.zip`, nomeado como `equipe_<nomes ou numero da equipe>.zip`.

Exemplo de compilação esperada:

```
gcc -Wall -o estoque main.c produto.c
./estoque
```

6. Apresentação

No dia 8, cada equipe apresentará o trabalho em até 10 minutos, com os 3 integrantes participando. A apresentação deve conter:

1. Breve explicação da struct utilizada e da lógica de alocação dinâmica do vetor de produtos.
2. Explicação de pelo menos uma função recursiva, destacando caso base e caso recursivo.
3. Demonstração ao vivo do programa em execução (compilar e rodar na hora).
4. Espaço para perguntas do professor e da turma.

7. Critérios de avaliação

A nota do trabalho será composta pelos seguintes critérios:

Critério	Peso
Funcionamento correto do programa (todas as operações do menu funcionam sem falhas)	30%
Uso correto de structs (organização dos dados do produto)	15%
Uso correto de alocação dinâmica (malloc/realloc/free, sem vazamento de memória)	20%
Implementação correta das funções recursivas exigidas	25%
Organização do código (modularização em .h/.c, comentários, nomes de variáveis)	10%

8. Observações finais

- Trabalhos idênticos entre equipes diferentes (cópia) serão zerados para todos os envolvidos.
- O trabalho será zerado se o participante da equipe não conseguir apresentar o trabalho, respondendo as perguntas do professor.